OPERATING SYSTEMS

Assignment 1

Smart Hospital Operating-System Evaluation

Student Mohamed Aslam H

Register number 192572147

Department CSE (AI)

Academic year 2025

Faculty Dr. Jayasakthi Velmurugan K

Course CSA0406 Operating Systems · CO1

Submission report · 12 tasks · Linux evidence attached in repository

Executive decision

Use a small hard real-time embedded OS (for example, a certified QNX configuration where certification is required, or FreeRTOS where the safety case permits) for bedside alarm acquisition, with bounded interrupt and scheduling latency. Use a managed handheld OS for tablets, supported desktop OSes for workstations, and a Linux/Unix database cluster with synchronous replication, quorum, fencing and tested failover. Use Linux with symmetric multiprocessing (SMP) for parallel MRI reconstruction. Connect hospitals through authenticated encrypted APIs and a distributed records service. Keep local alarm operation independent of WAN and cloud availability. For home devices, retain local alarm behavior and forward readings asynchronously; move shared records to a governed cloud service with identity federation and audit.

Task 1 - Requirements by environment

Ranks: 1 is the most important constraint. Reliability includes safe failure behavior; security is mandatory throughout, even where it is not ranked first.

Environment	Ranked needs	Reason and OS-history lesson
Bedside monitors	1 bounded response (<10 ms), 2 availability/fail-safe, 3 data integrity, 4 security, 5 low memory/power, 6 throughput	A missed alarm is a patient-safety event. A hard real-time scheduler must bound interrupt, dispatch and sensor-to-alarm latency under worst-case load, not merely show a good average. Batch systems maximize jobs but are unsuitable for immediate response. A small embedded RTOS avoids unrelated services.
Handheld tablets	1 battery, 2 security/privacy, 3 availability, 4 responsive UI, 5 wireless communication, 6 throughput	Mobile OS lessons include aggressive sleep, radio/power management, application sandboxing and managed updates. Battery savings must not defer a critical notification without an explicit clinical workflow.
Doctor/nurse workstations	1 confidentiality/integrity, 2 availability, 3 interactive response, 4 compatibility, 5 throughput	Time-sharing and personal-computer systems make interactive multi-user applications practical. Central identity, encryption, endpoint management and least privilege matter more than deterministic sub-10-ms response.
Patient-record database	1 integrity/durability, 2 availability, 3 access control/audit, 4 transactional throughput, 5 predictable latency, 6 maintainability	Mainframe/service OS ideas apply: concurrent transactions, isolation, resource accounting, backup and recovery. Redundancy is needed because a single OS update or hardware fault must not stop records access.
MRI/imaging compute	1 throughput, 2 data integrity, 3 availability, 4 scalable memory/I/O, 5 security	Multiprocessor systems and parallel programming are appropriate. MRI reconstruction can use queued work and exploit SMP; unlike alarms, it usually optimizes completion time rather than a hard deadline.
Partner-hospital link	1 confidentiality/integrity, 2 availability, 3 interoperability, 4 scalable throughput, 5 latency	Distributed systems must tolerate independent failures and network partitions. Exchange only through authenticated, audited interfaces; do not assume a remote partner is a trusted local process.

Conflicts: hard deadlines can conflict with throughput and memory sharing; encryption, logging and redundancy consume CPU, storage and energy; aggressive tablet sleep conflicts with immediate delivery; database maintenance conflicts with continuous service. Patient-safety-critical functions are local alarm acquisition, alarm signaling, trustworthy timestamps, and safe fallback behavior. Alarm generation must not depend on the database, a cloud service, or partner connectivity.

Single general-purpose OS alternative: One desktop/server distribution on all devices reduces image and administration diversity, but ordinary fair-share scheduling cannot guarantee a finite worst-case alarm delay under overload: the worst-case scheduling wait is not bounded by the product's normal-use latency. A general-purpose image and driver stack also consumes more RAM/flash and power than a tailored RTOS on monitors. Fewer OS families simplify

patching, but a universal configuration creates a common-mode failure and still requires per-device security, performance and update policy. It is rejected for the monitor deadline.

Task 2 - OS types

Environment	Selected type	Alternatives rejected / trade-off
Monitors	Embedded hard real-time, single-purpose	Soft RT and desktop systems offer no hard bound. Mainframe/cluster types are irrelevant at the sensor.
Tablets	Mobile/handheld	Desktop OS wastes power and exposes a broader service set.
Workstations	Desktop, centrally managed	Handheld lacks full clinical workstation capabilities; server OS is not optimized for interactive clinician use.
Database	Clustered server with failover	A standalone mainframe or large SMP server has a single-machine failure domain. A cluster costs more and needs quorum/fencing/replication engineering but limits downtime.
Imaging	Multiprocessor server, SMP Linux	AMP assigns fixed roles and can help specialized devices, but SMP is easier for general parallel reconstruction and load balancing. SMP has lock/contention scaling limits.
Partner link	Distributed system over secure network services	A single distributed OS spanning independent hospitals would impose unacceptable trust and failure coupling. Use interoperable services and explicit ownership boundaries.

Database alternative - one large multiprocessor: It avoids cluster replication and may have lower initial integration cost; however, motherboard, power, kernel and maintenance remain single points of failure. A cluster can fail over within seconds if health checks, quorum, fencing and replicated state are configured and practiced; failover is not zero-loss or automatic without a consistency design. For parallel workloads, ideal CPU time can approach T_1/p with p processors (Amdahl's law limits real speedup); memory capacity is the node's installed RAM, whereas a cluster's RAM is generally not one automatically shared address space. Cost comparison depends on vendor, licensing, storage and support quotes and is not asserted numerically here.

Task 3 - OS structures

Structure	Reliability / performance / update assessment
Monolithic	Fast direct calls; large trusted kernel and faulty in-kernel driver can crash or corrupt the whole system.
Layered	Clear interfaces ease reasoning and testing; extra layer crossings can add overhead and rigid boundaries can constrain design.
Microkernel	Small privileged core, isolated services and message passing improve fault containment; IPC/context switches add latency and require careful worst-case analysis.
Modular	Loadable modules combine fast kernel calls with extensibility; modules still run privileged and can destabilize the kernel.
Hybrid	Combines kernel and service ideas to balance compatibility and performance; complexity can weaken the benefits of separation.

Linux is a monolithic kernel with loadable modules; Windows NT is hybrid; Android uses the Linux kernel with Android user-space framework/services; QNX is a microkernel RTOS, while FreeRTOS is a small real-time kernel rather than a full POSIX desktop OS [6][7]. Verify the exact target version/configuration before making safety or certification claims.

Choices: For a monitor, use a microkernel or small RTOS design with critical sensor/alarm paths isolated from optional services, provided measured worst-case IPC and interrupt latency fit the 10-ms budget. For the database, use a mature

modular kernel (e.g., Linux) for drivers, observability and storage/network performance, backed by a tested HA cluster; kernel modularity does not itself deliver HA.

Monolithic monitor alternative: A sensor service request through a syscall is typically one trap and kernel dispatch; a microkernel design may add IPC and a context switch or two. Those fixed costs are hardware and configuration dependent and must be measured against the deadline; they are not safely represented by generic nanosecond claims. A monolithic kernel can reduce memory overhead and call overhead, but a faulty driver has a broader failure domain. QNX documents user-space isolation of drivers and services as a core microkernel property [8]. Microkernel memory overhead includes servers and message buffers; isolation can justify that cost for patient safety.

Task 4 - Services

Service	Monitor	Tablet	Workstation	Database	Imaging	Partner link
UI	Minimal alarm/status	Touch UI	Full desktop	Admin/monitor only	Operator/queue	API/service only
Program execution	Fixed trusted tasks	Clinical apps	Clinical apps	DB engine/agents	Reconstruction workers	Gateway services
I/O	Sensor, alarm, watchdog	Display, radio, camera as needed	Display, keyboard, network	Storage/network	Scanner, fast storage/network	Network interfaces
File system	Minimal config/log	Encrypted managed storage	Encrypted managed storage	Journals, data, backups	Scratch and image archive	Queue/audit logs
Communication	Isolated local bus	Wi-Fi/cellular/VPN	Hospital network	Cluster and client links	Scanner/cluster	TLS APIs
Error detection	Watchdog, range checks, alarm	Health/MDM	Endpoint monitoring	ECC, checksums, replication	ECC/checksums	Validation/retries
Allocation/accounting	Static priority/bounded CPU	OS-managed energy/resources	Multi-user scheduling	QoS, quotas, telemetry	Scheduler/GPU/CPU	Rate limits/quotas
Protection/security	Secure boot, signed updates, least privilege	Device encryption, sandbox, MFA	MFA, least privilege	RBAC, audit, encryption, separation of duties	Least privilege, audit	Mutual auth, scoped consent, audit

Remove from monitor: general-purpose shell, package manager, unused network daemons, broad UI, printing, general user accounts and nonessential filesystems. Keep only services needed for safe sensing, alarms, bounded logs, time, secure update and diagnostics. Database essentials include virtual memory, concurrency, durable filesystems, transactional synchronization, networking, audit, privilege separation, backup/restore and error reporting. Patient records are isolated using authenticated identities, database row/column policy where needed, process address spaces, least privilege, encrypted transport/storage, audit, and tested backup/restore. OS process isolation alone does not replace database authorization.

Desktop OS on tablets alternative: Extra services increase RAM/flash footprint, startup work, background wakeups and attack surface; sleep/radio settings may reduce battery duration. Exact MB/minutes require measurement on the chosen hardware and workload. A mobile OS is selected because power-aware scheduling and a narrower managed app profile better fit a shift-long device; full desktop compatibility may be preferable only if a clinical app requires it and battery testing supports it.

Task 5 - System calls

System-call categories: process control; file management; device management; information maintenance; communication; and protection. For `read(fd, buf, count)`, a C library wrapper places arguments according to the ABI and executes the syscall instruction/trap [1]. CPU changes privilege to kernel mode and enters the syscall handler; kernel validates descriptor, permissions and user buffer, requests data from cache/device (possibly sleeping), copies available

bytes to user space, sets return value/error, then returns to user mode. A read may return fewer bytes than requested [1]. Parameters are commonly in registers, or a pointer to a memory block; some ABIs/stubs also use the stack.

`fopen`/`fread`/`fwrite` are C library interfaces. `fopen` usually makes an `openat`-family syscall. Buffered `fread` can satisfy many reads from a library buffer; with n bytes and buffer B, data calls are roughly ceil(n/B), plus open/close and possible metadata calls, versus about ceil(n/b) for direct reads with b-byte chunks. Library buffering costs approximately B bytes per stream plus bookkeeping and may defer errors until flush/close. Direct syscalls avoid stream buffering but require short-read/short-write handling.

Linux evidence: evidence/task06-strace-summary.txt is the actual Ubuntu 24.04 strace -c output [2] for the Task-06 executable, which is also the real read/write workload discussed here. It reports 10,027 read calls overall for the complete process (including runtime and file setup), while the copy loops themselves print 10,001 + 21 + 4 = 10,026 reads. This small difference is startup/runtime I/O outside the three measured copy loops. The latest full workflow output is in run 5: https://github.com/Md-Aslam-31/CSA0406-Operating_System/actions/runs/41692084440.

Task 6 - Cost of system calls and completed program

`Task-06/copy_file.c` fills the loop, retries interrupted calls, handles partial writes, validates buffer size and checks errors. For the 10,000-byte input, with one final EOF read counted, expected calls are ceil(10000/b)+1: 10001 for b=1; 21 for b=512; 4 for b=4096. The program compiled with GCC 13.3.0 on Ubuntu and produced those exact counts. The same output was also reproduced locally with MSYS2 GCC 16.1.0. Linux `strace -c` reports process-wide syscall totals, including startup and stdio, rather than only the function's read counter. *The latest CI run also verified output-byte equality, zero and oversized buffers, missing input, and invalid destination.*

Buffer	Copy reads (measured by program)	Approx. user buffer	Assessment
1 B	10,001	1 B	Extreme syscall overhead; reject.
512 B	21	512 B	Far fewer calls; still more dispatches.
4096 B	4	4096 B	Strong reduction with tiny memory cost for this workload.

For backup, begin with 64 KiB-1 MiB buffered I/O and benchmark throughput, memory pressure and device behavior on actual Linux hardware; 4 KiB wins among the supplied sizes by syscall count but cannot be called universally fastest without timing. A larger buffer reduces call count to O(n/b), while memory per copy remains O(b); copying itself is O(n). `mmap` can reduce explicit read/write calls and copies [9], but still incurs page faults, mapping calls, page-cache work and possible writeback; touching a huge mapping uses virtual address space O(n), while resident memory is workload-dependent and constrained by cache eviction. For bounded-memory backup, chunked I/O is easier to limit and recover from.

Task 7 - Process control

`fork()` creates a child (typically using copy-on-write) [3]; `exec()` replaces the child image [4]; `wait()`/`waitpid()` reaps termination and returns status [5]; `exit()` flushes user-space streams and terminates, whereas `_exit()` terminates without stdio flushing. The shell forks, child execs the requested command, parent waits/reaps and reports status. The completed `Task-07/shell_jobs.c` implements `expr 6 * 7` and a child exiting 3; it ran on Ubuntu 24.04 and produced the expected output preserved in `evidence/task07-output.txt`. *CI also verified exec failure status 127 and demonstrated the duplicated buffered line after fork when exit() flushes.*

Process sketch:

```
shell (parent) --fork--> child --exec(expr)--> prints 42, exits 0
       |--waitpid-----------------------------^ reports status 0
shell --fork--> child --_exit(3)
       |--waitpid-----------------------------^ reports status 3
```

A zombie is a terminated child whose parent has not yet collected status; `wait` reaps it. An orphan has a living process after its parent exits and is reparented to the system's designated subreaper/init, which can reap it. `fork` plus `exec` allows the shell to set up redirection, pipes and environment between creation and program replacement; `posix_spawn` can combine common steps efficiently. `fflush` before `fork` prevents a buffered line copied into both processes from being emitted twice on later flush. With redirected output, `exit(3)` flushes the inherited duplicate buffer; `_exit(3)` does not. Removing the second flush before forking the second child therefore duplicates the buffered "job 2" line when the child calls `exit`.

Alternative: `posix_spawn` may lower start-up overhead for simple commands; performance for 1,000 launches must be benchmarked on the same host. Threads share address space and most process resources, reducing creation/memory cost, but commands then share mutable state and are not isolated; each thread needs its own stack. Processes have separate virtual address spaces (initially copy-on-write) and stronger fault/security isolation. A database shell should prefer process isolation for arbitrary commands.

Task 8 - CPU states and protection

User mode restricts privileged instructions; kernel mode may control devices, page tables and interrupt state. A mode bit/state distinguishes privilege. Hardware interrupts signal external events; traps/exceptions arise synchronously from instructions (including syscalls); timer interrupts return control from a running process so the scheduler can enforce time slices/priorities. If arbitrary monitor code could disable interrupts, it could suppress sensor handling and alarm deadlines indefinitely, so only trusted kernel code can perform that operation. MMU page tables and privilege checks prevent one process from reading another's memory; DMA requires IOMMU/device isolation as well.

Polling alternative: with polling interval P, worst-case detection wait is approximately P plus processing latency; interrupt-driven detection is bounded by interrupt masking/dispatch latency plus handler time. Polling every 1 ms may meet a 10-ms target but consumes CPU and energy even when readings do not change; a slower interval saves energy but raises worst-case detection delay. An interrupt path can lower idle cost, but must be measured and protected from interrupt storms. The monitor design uses interrupts, bounded high-priority handling and a watchdog, while reserving polling for diagnostics or where sensor hardware requires it.

Task 9 - Memory hierarchy and I/O

Typical order-of-magnitude values vary by hardware: CPU registers ~sub-ns to a few ns; L1 cache ~1 ns, L2 several ns, L3 tens of ns; DRAM ~50-150 ns; NVMe SSD ~tens-hundreds of µs; HDD ~5-15 ms; network storage ~sub-ms to tens of ms depending on network/load. Capacity generally increases as speed decreases. These are illustrative ranges, not benchmark claims.

Data	Placement and rationale
Active monitor values/alarm state	Registers/cache/DRAM in local controller; bounded durable event log in flash. Never block alarm on remote storage.
Frequently accessed patient index/records	Database DRAM buffer/cache backed by replicated durable SSD/storage; cache requires invalidation/consistency controls.
MRI scan in progress	DRAM buffers and high-throughput local NVMe/scratch; archive validated images to managed durable storage.
Long-term records/images	Encrypted replicated storage with offline/immutable backup and tested restore.

Programmed I/O ties CPU to every transfer and suits simple low-rate controls. Interrupt-driven I/O avoids busy waiting and suits moderate event rates. DMA transfers blocks with less CPU intervention and suits high-volume MRI acquisition; use bounded buffers, completion interrupts, checksums and IOMMU protection. For a 500 MB image at 4 bytes/word, interrupt-per-word implies 125,000,000 interrupts. At even 1 µs handler overhead each, the interrupt overhead alone is 125 seconds; at 5 µs, 625 seconds. DMA instead needs a small number of block completions, plus setup/interrupt overhead. Interrupt-per-word also requires buffering/processing each word; DMA needs one or more bounded large

buffers and descriptors. Caching speeds repeated lookups by avoiding slower storage, but stale clinical data must be prevented with transactional invalidation/consistency rules.

Task 10 - Measurements

Ubuntu 24.04 run evidence is in evidence/. GCC 13.3 compiled both programs with -std=c11 -Wall -Wextra -pthread; Task-06 and Task-07 outputs matched expected behavior. The latest timed trial measured 1,000 sequential fork+wait children at 0.137441 s and creation/join of 1,000 no-op pthreads at 0.043998 s. In this run threads were about 3.2x faster, consistent with sharing the process address space, but this is one short runner sample and must not be generalized; scheduler load, CPU, libc and kernel affect it. The accompanying /usr/bin/time -v reports 0.18 s elapsed and 9,896 KiB maximum RSS for the combined benchmark. Task-06 total process timing was 0.01 s (user 0.00, system 0.01) for all three buffer copies; this timer is too coarse to distinguish individual sizes. The 1,000-process and 1,000-thread comparison includes wait/join costs. Processes provide stronger isolation and initially share pages copy-on-write; threads share code/data/address space but each has a stack and synchronization requirements.

Task 11 - Complete recommendation

Tier	OS plan	Services / protection / I/O	Main risk and control
Bedside	Minimal hard RTOS	Fixed-priority tasks, sensor interrupts, watchdog, signed updates, minimal logs; no cloud dependency	Deadline miss/driver fault; worst-case timing validation, independent alarm path, fail-safe state, clinical validation
Mobile	Managed mobile OS	Encrypted storage, sandbox, MFA, MDM, Wi-Fi/cellular TLS	Lost device/radio outage; remote revocation, local workflow, battery/notification testing
Clinical desktop	Managed desktop OS	MFA, endpoint controls, least privilege, audit, encrypted network	Credential theft/malware; patching, segmentation, monitoring
Database	Clustered Linux server	RBAC, audit, encrypted storage/network, checksums, synchronous replication as required, quorum/fencing, backups	Split brain, correlated failures, bad update; tested failover/restore, staged patching, independent failure domains
Imaging	SMP Linux compute nodes	Parallel job scheduler, DMA-capable I/O, ECC, checksum and access controls	Contention/driver/scanner faults; queueing, resource limits and validated data pipeline
Inter-hospital	Distributed services	Mutual TLS, scoped APIs, audit, retries/idempotency, versioned schema	WAN partition/data mismatch; local autonomy, reconciliation and governance

Scale from one ward to 500 monitors/120 tablets/80 workstations by provisioning and managing devices centrally while retaining local monitor scheduling. For three hospitals, distribute identity and records interfaces, define authoritative record ownership, and test WAN partitions, replication lag and recovery. Risks include common-mode updates, vendor support/certification constraints, human-factor alarm fatigue, cluster misconfiguration and cost. Mitigate with staged rollouts, rollback, hazard analysis, operational drills, access review and measurements on target hardware.

Virtualizing everything on two large servers may improve consolidation and snapshots, but monitor timing can be affected by hypervisor scheduling, device emulation and noisy neighbors; only a real-time hypervisor with reserved CPUs and validated passthrough might qualify, and bedside alarms should remain local. Virtualization consumes host RAM/CPU for the hypervisor, guests and reserved headroom. Two hosts still share facility/network/storage risks and can fail together; HA requires independent power, storage/network paths and tested recovery. Keep safety-critical monitors physical/local; virtualization is appropriate for many server workloads with measured overhead and redundancy.

Task 12 - Day-4 design challenge: MODIFY

A. Impact: Home readings add hostile/unreliable WAN paths, heterogeneous devices, remote updates, identity and consent concerns. Cloud records add distributed consistency, availability-zone failure, data residency/retention and partner access control. Monitor RTOS type remains suitable; home-device security, mobile/network services and database topology are affected.

B. Original design: Local hospital monitors still meet their intended deadline if alarms remain locally generated. A tablet or home device using a remote cloud round trip cannot guarantee a 10-ms alarm due to variable internet delay and outages. The clustered database concept remains useful but a single hospital-local cluster alone does not provide multi-hospital cloud availability. Security must extend to device identity, transport, tenant separation, audit and revocation.

C. Adaptation: Modify rather than replace. Home units use embedded RTOS or constrained mobile/embedded OS with local threshold detection, buffering and authenticated signed updates. Send readings over mutually authenticated TLS to an ingestion gateway; queue, validate, timestamp and deduplicate records. Host the shared record service across independent cloud failure domains with encrypted backups, regional recovery, tenant/role-based access, audit and explicit consistency semantics. Hospital bedside alarms remain local. Partner systems use versioned APIs and scoped identity federation; allow offline workflows and later reconciliation.

D. Defence: This preserves deterministic local behavior and existing desktop/imaging tiers while adding distributed protections where the changed trust boundary requires them. The trade-off is greater identity, monitoring, governance and operational complexity, plus cloud/network cost and possible replication lag. WAN outages must reduce remote visibility, not disable local alarm response. Validate remote alert latency as a service objective, not a hard local-control guarantee; perform threat modeling, penetration testing, disaster recovery and clinical usability testing before deployment.

References

1. Linux man-pages project, `read(2)`, https://man7.org/linux/man-pages/man2/read.2.html.
2. Linux man-pages project, `strace(1)`, https://man7.org/linux/man-pages/man1/strace.1.html.
3. Linux man-pages project, `fork(2)`, https://man7.org/linux/man-pages/man2/fork.2.html.
4. Linux man-pages project, `execve(2)`, https://man7.org/linux/man-pages/man2/execve.2.html.
5. Linux man-pages project, `wait(2)`, https://man7.org/linux/man-pages/man2/waitpid.2.html.
6. AWS, FreeRTOS Kernel Fundamentals, https://docs.aws.amazon.com/freertos/latest/userguide/dev-guide-freertos-kernel.html.
7. FreeRTOS, Task Scheduling, https://github.com/FreeRTOS/FreeRTOS-Kernel-Book/blob/main/ch04.md.
8. QNX, Microkernel Architecture, https://qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.sys_arch/topic/intro_MICROKERNELARCH.html.
9. Linux man-pages project, `mmap(2)`, https://man7.org/linux/man-pages/man2/mmap.2.html.
10. Canonical, Install Ubuntu on WSL 2, https://ubuntu.com/wsl/docs/latest/howto/install-ubuntu-wsl2/.

The assignment PDF and evaluation key supplied for this work are the controlling rubric. Quantitative formulas (call counts, complexity, interrupt estimate) are derived explicitly in text. Timing values are actual single-run CI measurements and are not universal benchmarks. No vendor certification or price is asserted without a target system. Linux program outputs and measurements were executed in the Ubuntu 24.04.5 CI workflow; local Windows WSL installation remains pending an elevated install and restart.